# Weight Divider

Bilkent University Electrical and Electronics Engineering Department

EEE102 Term Project

Emre ORAL

22003050

Section 3

**YouTube:**

https://youtu.be/-3d0ImhqhpA

**Objective:**

The aim of this project is to design a weight meter that can measure up to a certain weight and to classify the results of the weight meter according to the weight we have determined in our mind. (For example, if the value we get from the weight meter is 6 grams and the value we have determined is 10 grams, this system pushes the weight to the right if it is greater than 10 grams, to the left if it is less than 10 grams, and the classification is completed.)

**Methodology:**

In the first part of the project, I had to measure weight. I used a weight sensor, a weight sensor amplifier (hx711) and finally an Arduino uno to perform this operation. In order to get the most accurate values, after sticking the weight sensor on the board with half of it out, I provided the connections with jumper cables and finally connected the Arduino to the computer. At this part, I converted the values I received from the weight sensor as analog signals into digital signals. To use these digital signals, I created a system that displays a calibrated weight value for 2 seconds using the hx711 module on the arduino. In the second part of my project, I had to write a VHDL code to classify this weight value and the weight value I determined in my head, and with this code, I had to run the servo motors in a coordinated manner with basys3. For this process, I first wrote a code that allows us to rotate a servo motor with six different angle values. After that, I wrote a comparator code that allows us to compare two binary numbers. After these operations, I connected the output of the comparator code to the input that rotates the servo motor ninety degrees with the help of buffer. I used two different servo motors for two different directions. Finally, I did the connecting with jumpers and set up the assembly. My project was now ready to run.

Design Specifications:

There are 4 inputs and 2 outputs as follow:

```
ain, bin: in  std_logic_vector(3 downto 0);
```

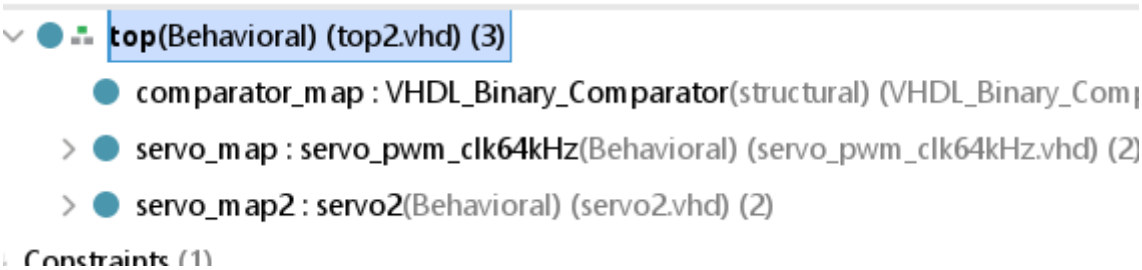
reset: in std_logic;

clk: in std_logic;

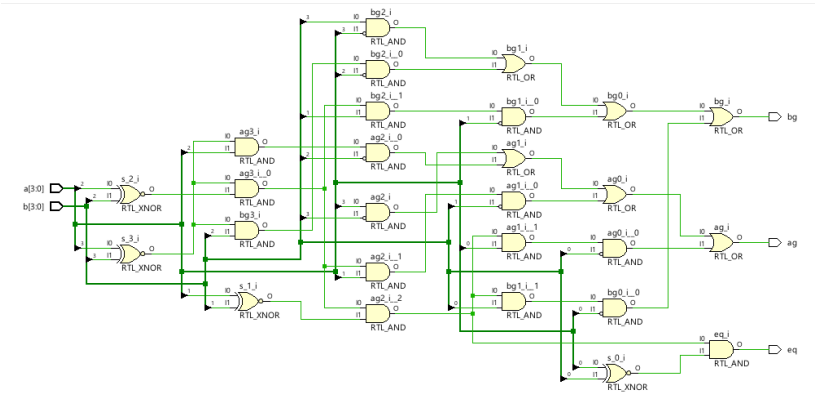
servo: out std_logic;

servoo: out std_logic

The design includes two servo drivers and one four-bit binary comparator

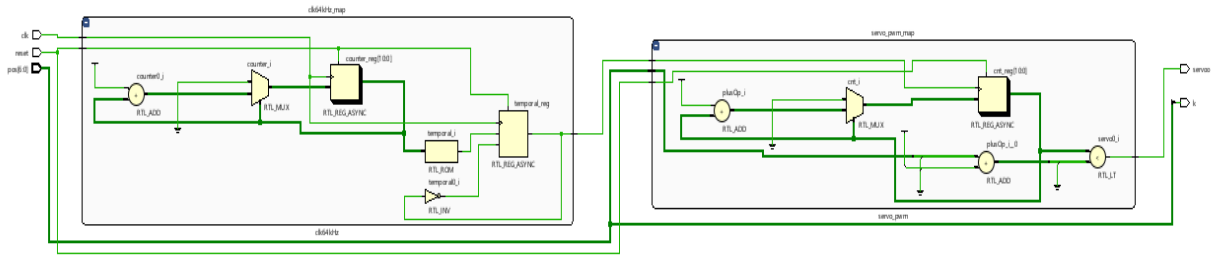



The comparator map has 2 inputs and 3 outputs inside, the inputs are four-bit binary numbers, and the outputs are: weight greater than scale, scale greater than weight and they are equal.



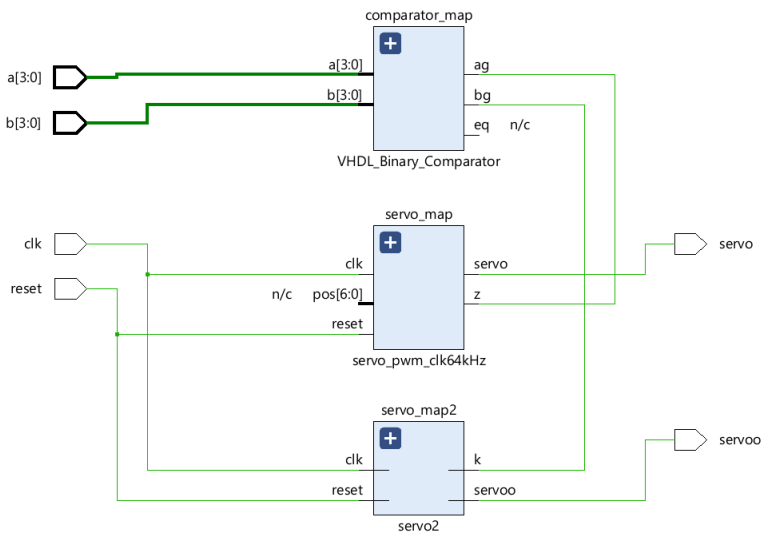
To make this comparison happen, I used xnor gates that compare the value of each digit. The results obtained are finally checked with the or and and gates, and one of our outputs gives the value 1 according to the result.

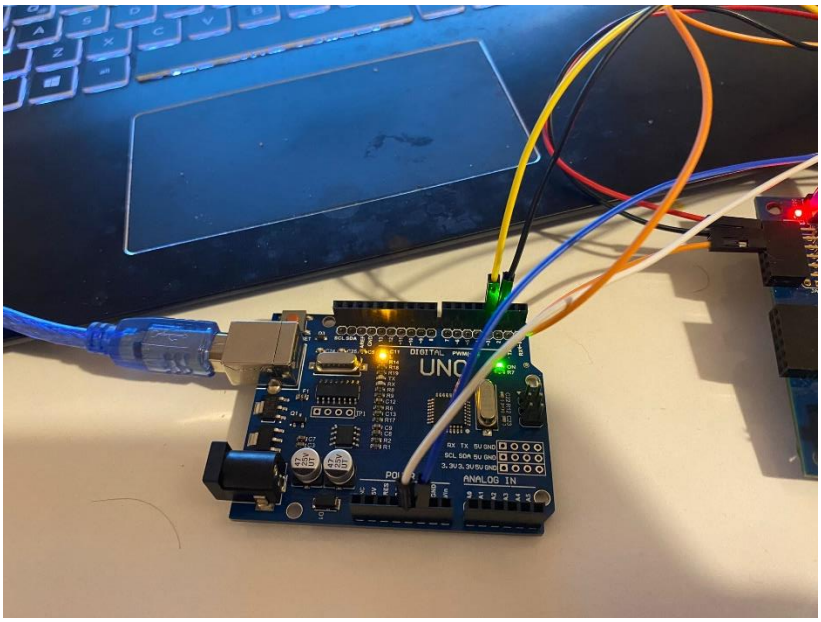
After these processes, it was time for the codes required for the servo to work. There are 2 different sources of code under the servo map 1 module. One of these codes enabled the servo motor to work with 64kHz clock, and the other allowed us to obtain the output signal.

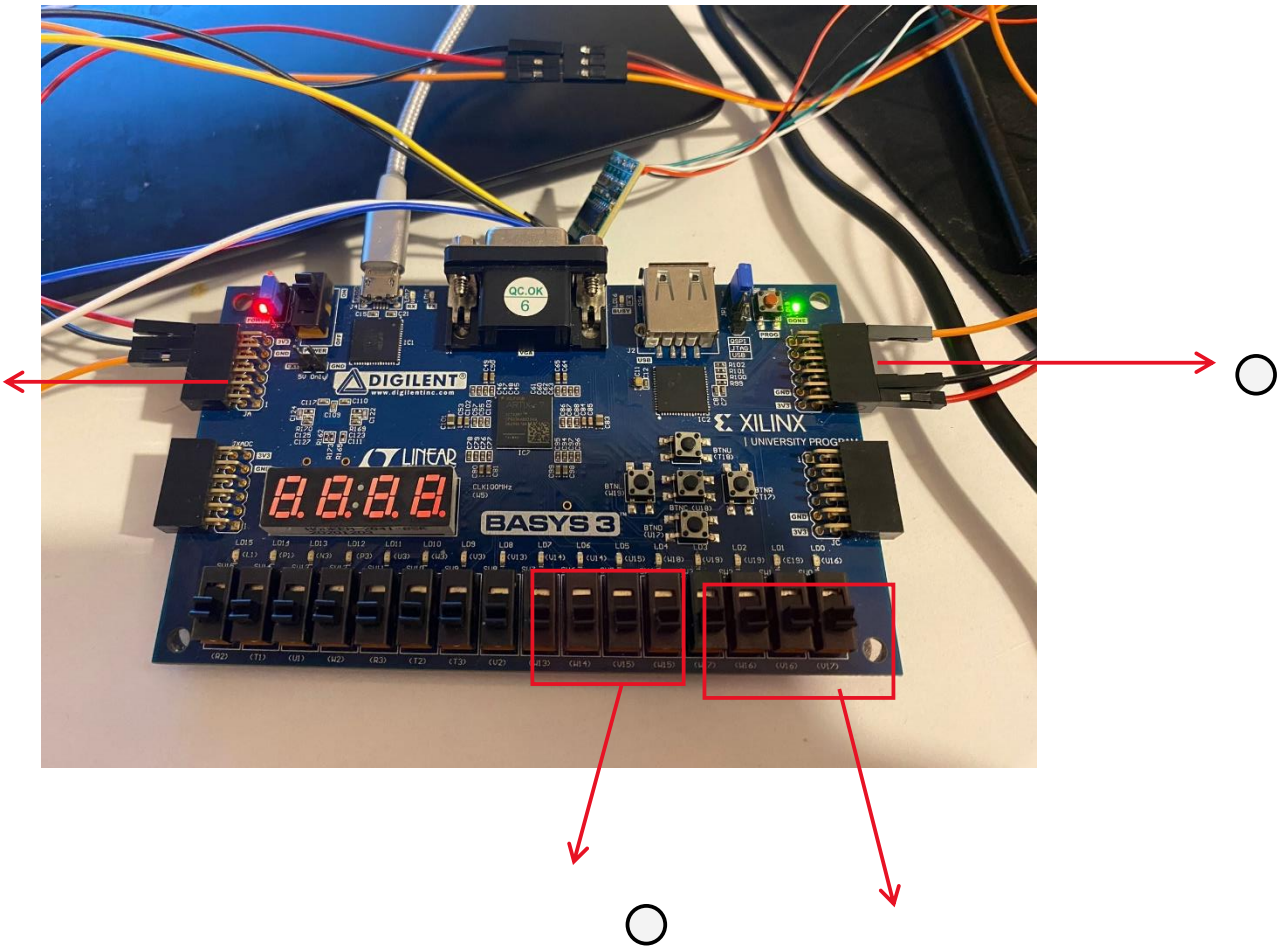


My code has 7 different position inputs, each of which allows us to obtain different angle values, but it will be enough for us to use the 90 degree position input. Apart from these inputs, we have clock and reset inputs and I have 1 output that enables the servo to work.

As a final step, I had to write a top module to combine these two modules. First, I created the 2nd servo motor module using the same codes, then I defined each component in the top module. The critical part of this step was to extract the 6th position vector of the servos with the help of buffer and connect the scale outputs here, where the scale is greater than the weight and the weight is large.

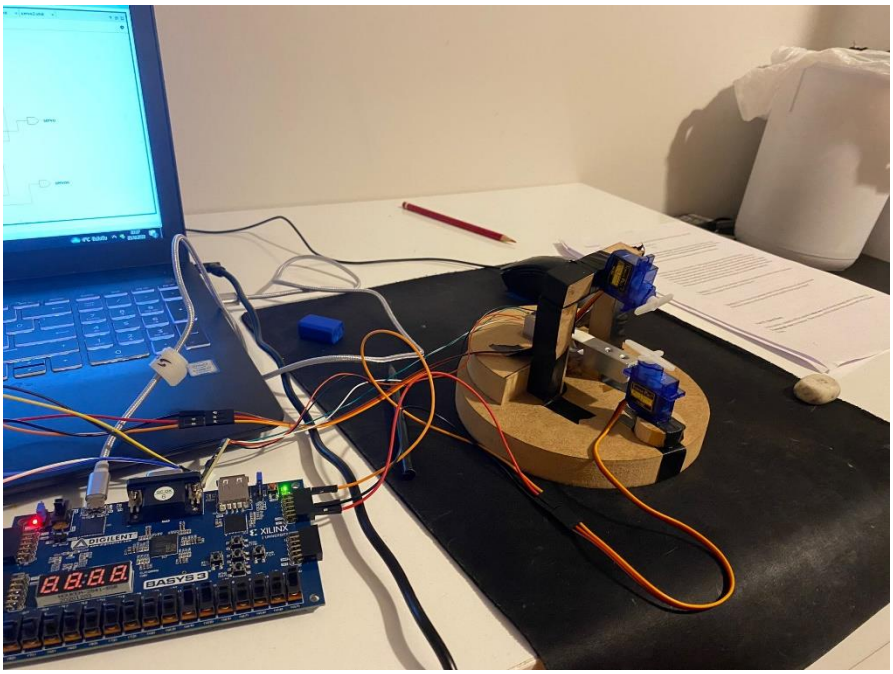



ANALOG IN

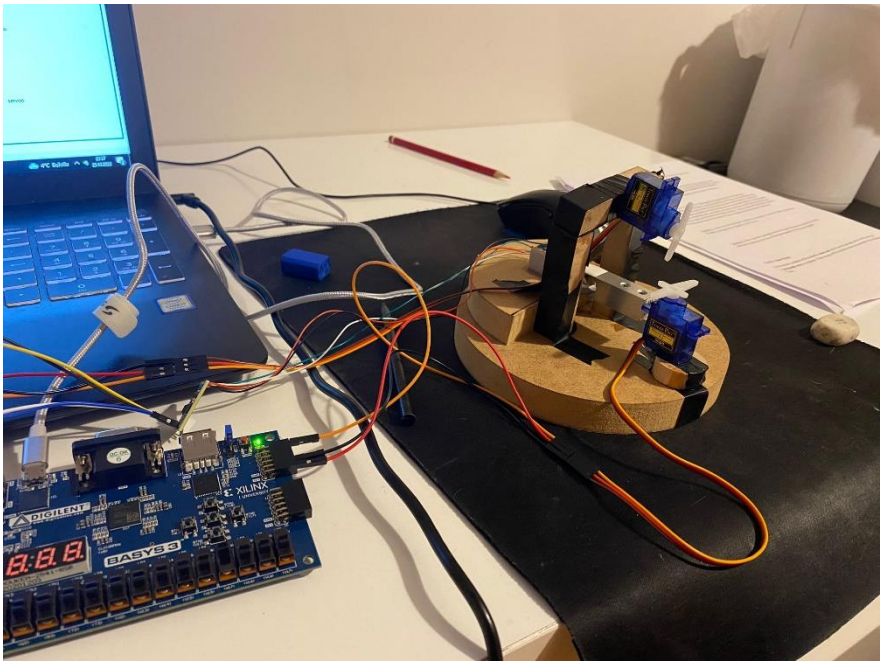

DIGILENT
www.digilentinc.com
LINEAR
XILINX
UNIVERSITY PROG
BASYS 3
QC.OK
6

**Results:**

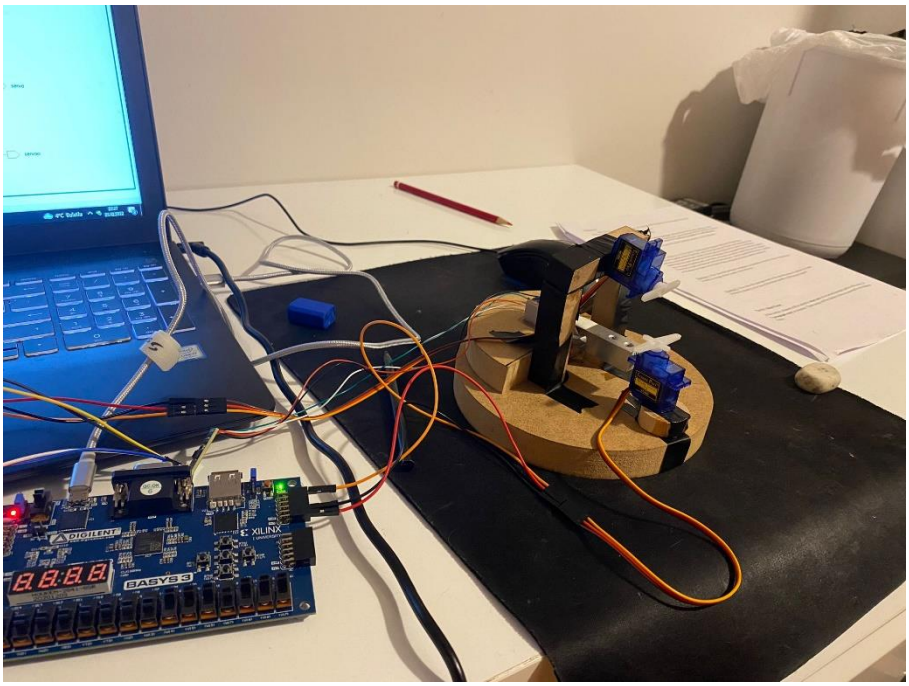
There are 3 position outputs we can get here.



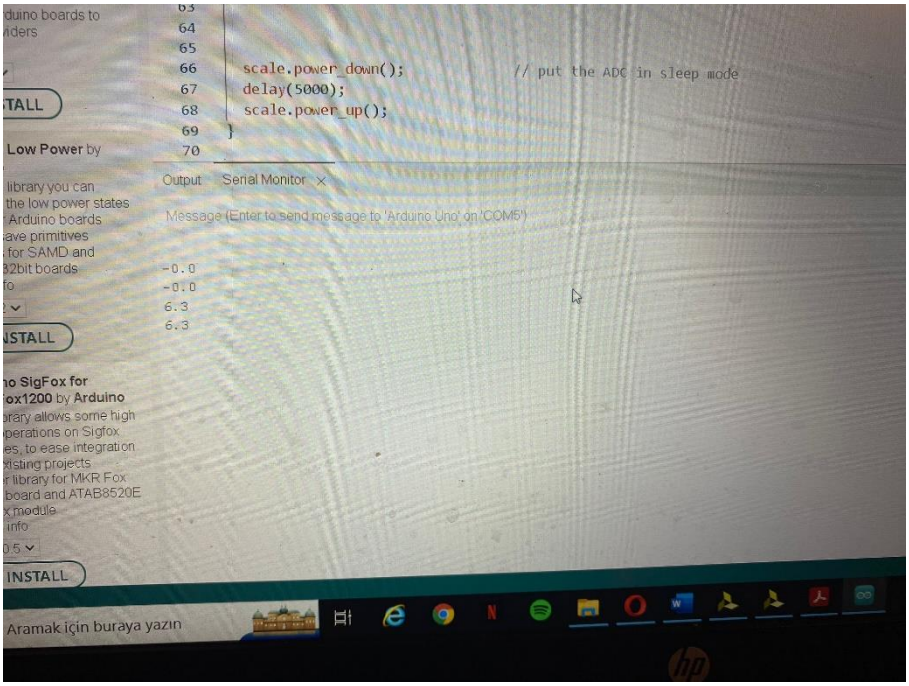

The Weight = 0001  The Scale = 0110




The Weight= 1000  The Scale= 0000




The Weight= 0111 The Scale= 0111




The results of weight sensor

**Conclusion:**

The main purpose of the project was to divide a weight into two parts according to the scale and the rules I set, and I achieved this goal. At the beginning of the project, I was quite ignorant about signal transition from analog to digital, but as a result of my research, I learned a lot of new technological tools and the conveniences they provide for me. The HX711 was one of them, and it was quite a useful tool. After that, I learned how to use Arduino for weight measurement and researched many different modules. All of these modules were designed for the HX711, but they had minor differences, and it was really important which one to use. After this process, my new goal was to understand the working principle of servo motors and integrate it into basys3. I achieved this and developed a working system with the help of VHDL code. While creating this system, I read many articles about servo motors and overcame the difficulties I faced thanks to what I learned from them. Finally, I now had a properly working system. It was a good feeling to hold this project that you worked for so many days and I learned a lot while doing it.

**References:**


1) https://github.com/bogde/HX711


**Appendix:**

**TOP MODULE:**

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;


entity top is

   port(

      ain, bin : in  std_logic_vector(3 downto 0);

      reset: in std_logic;

      clk: in std_logic;

      servo: out std_logic;
```

```
        servoo: out std_logic
    );
end top;

architecture Behavioral of top is
    component servo2
        port(
            clk  : IN  STD_LOGIC;
            reset: IN  STD_LOGIC;

            servoo: OUT STD_LOGIC;

            k   : buffer std_logic

        );
    end component;
    component servo1
        port(
            clk  : IN  STD_LOGIC;
            reset: IN  STD_LOGIC;

            servo: OUT STD_LOGIC;
            z : buffer std_logic



        );
```

```
    end component;

    component Fourbitcomparator
        port(
            ain, bin : in  std_logic_vector(3 downto 0);
            agreater ,bgreater : out std_logic
        );
    end component;

    signal bgreater : std_logic:= '0';
    signal agreater : std_logic:= '0';
begin
    comparator_map: Fourbitcomparator port map(
        ain, bin, agreater, bgreater
    );
    servo_map: servo1 port map(
        clk  , reset , servo, agreater
        );
    servo_map2: servo2 port map(
        clk , reset, servoo, bgreater
        );

end Behavioral;
```

xxxxxxxxxxxxxxxxxxxxxxxxxx

**4-bit Binary Comparator:**

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity Fourbitcomparator is

  port (
    ain, bin : in  std_logic_vector(3 downto 0);
    agreater, bgreater, eq : out std_logic);

end Fourbitcomparator;

architecture structural of Fourbitcomparator is
signal s : std_logic_vector(3 downto 0);        -- intermediate signal
begin

s(0)<= ain(0) xnor bin(0);
s(1)<= ain(1) xnor bin(1);
s(2)<= ain(2) xnor bin(2);
s(3)<= ain(3) xnor bin(3);

eq<=s(3) and s(2) and s(1) and s(0);

agreater<=(ain(3) and (not bin(3))) or (s(3) and ain(2) and (not bin(2))) or (s(3)
and s(2) and ain(1)and (not bin(1))) or (s(3) and s(2) and s(1) and ain(0) and (not
bin(0)));
```

```
bgreater<=(bin(3) and (not ain(3))) or
   (s(3) and bin(2) and (not ain(2)))or
   (s(3) and s(2) and bin(1)and (not ain(1))) or
   (s(3) and s(2) and s(1) and bin(0) and (not ain(0)));

end structural;
```

xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

**SERVO TOP:**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity servo1 is
   PORT(
      clk  : IN  STD_LOGIC;
      reset: IN  STD_LOGIC;
      position  : IN  STD_LOGIC_VECTOR(6 downto 0);
      servo: OUT STD_LOGIC;
      z   : buffer std_logic

   );
end servo1;

architecture Behavioral of servo1 is
   COMPONENT servoclock
      PORT(
         clk    : in  STD_LOGIC;
```

```
        reset  : in  STD_LOGIC;
        clockout: out STD_LOGIC
      );
    END COMPONENT;

    COMPONENT servo_pwm
      PORT (
        clk   : IN  STD_LOGIC;
        reset : IN  STD_LOGIC;
        position   : IN  STD_LOGIC_VECTOR(6 downto 0);
        servo : OUT STD_LOGIC
      );
    END COMPONENT;

    signal clockout : STD_LOGIC := '0';
  begin
    servoclock_map: servoclock PORT MAP(
      clk, reset, clockout
    );

    z <= position(5);
    servo_pwm_map: servosignalPORT MAP(
      clockout, reset, position, servo
    );
  end Behavioral;
```

xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

**SERVO CLOCK:**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity servoclock is
   Port (
      clk    : in  STD_LOGIC;
      reset  : in  STD_LOGIC;
      clockout: out STD_LOGIC
   );
end servoclock;

architecture Behavioral of servoclock is
   signal temporal: STD_LOGIC;
   signal counter : integer range 0 to 1560 := 0;
begin
   freq_divider: process (reset, clk) begin
      if (reset = '1') then
         temporal <= '0';
         counter  <= 0;
      elsif rising_edge(clk) then
         if (counter = 1560) then
            temporal <= NOT(temporal);
            counter  <= 0;
         else
            counter <= counter + 1;
```

```
        end if;
      end if;
   end process;

   clockout <= temporal;
end Behavioral;
```

xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

**SERVO SİGNAL:**

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity servosignalis
   PORT (
      clk   : IN  STD_LOGIC;
      reset : IN  STD_LOGIC;
      position   : IN  STD_LOGIC_VECTOR(6 downto 0);
      servo : OUT STD_LOGIC
   );
end servo_pwm;

architecture Behavioral of servosignal is
   -- Counter, from 0 to 1279.
   signal counttt : unsigned(10 downto 0);
   -- Temporal signal used to generate the PWM pulse.
   signal pww: unsigned(7 downto 0);
```

```
begin
    -- Minimum value should be 0.5ms.
    pww <= unsigned('0' & position) + 32;
    -- Counter process, from 0 to 1279.
    counter: process (reset, clk) begin
        if (reset = '1') then
            counttt <= (others => '0');
        elsif rising_edge(clk) then
            if (counttt = 1279) then
                counttt <= (others => '0');
            else
                counttt <= counttt + 1;
            end if;
        end if;
    end process;
```